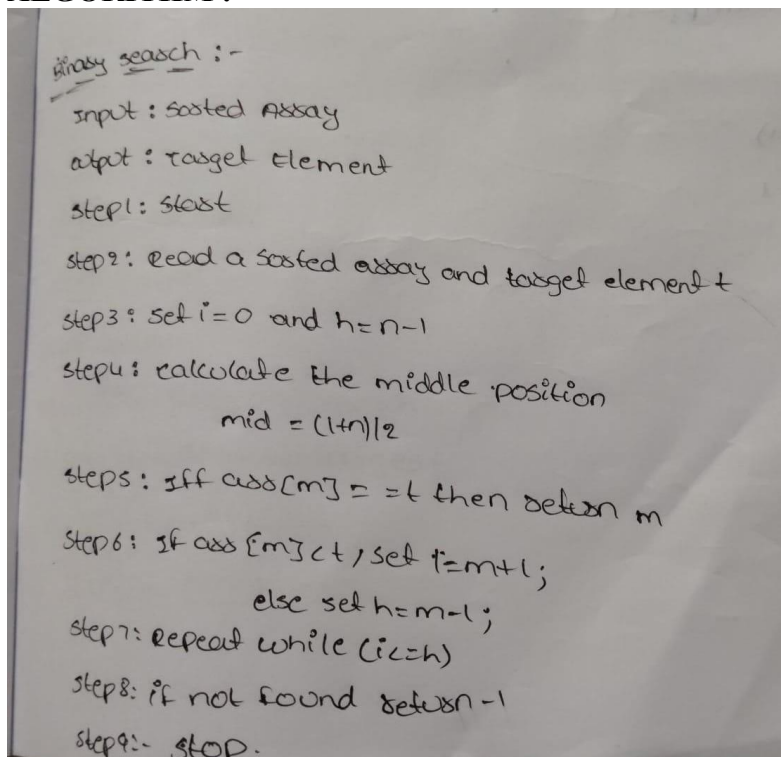


PRACTICAL - 2

AIM : Implementation and Time analysis of linear and binary search algorithm

BINARY SEARCH ALGORITHM :



CODE:

```
#include <iostream>
using namespace std;

int main()
{
    int n, arr[100], key;
    int low, high, mid;

    cout << "Enter size of array: ";
    cin >> n;

    cout << "Enter sorted array elements: ";
    for(int i = 0; i < n; i++)
        cin >> arr[i];
```



```
cout << "Enter element to search: ";
cin >> key;

low = 0;
high = n - 1;

while(low <= high)
{
    mid = (low + high) / 2;

    if(arr[mid] == key)
    {
        cout << "Element found at position " << mid + 1;
        return 0;
    }
    else if(arr[mid] < key)
    {
        low = mid + 1;
    }
    else
    {
        high = mid - 1;
    }
}

cout << "Element not found";

return 0;
}
```

OUTPUT:

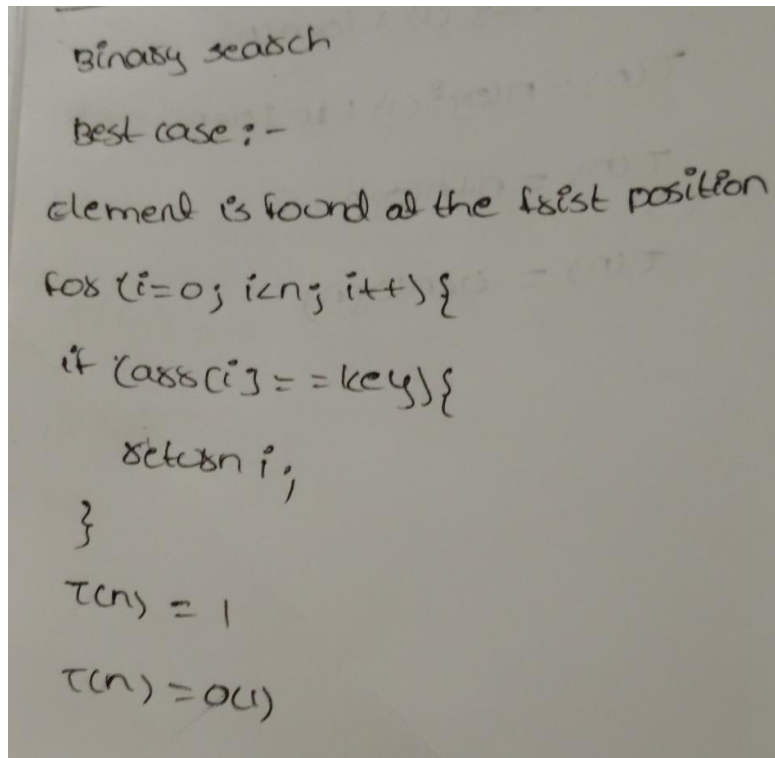
Output

```
Enter size of array: 5
Enter sorted array elements: 2
5
7
35
89
Enter element to search: 5
Element found at position 2
```



TIME COMPLEXITY:

Best case :





Average case : Key is found after several divisions

```
average case :-  
low = 0; - 1  
high = n-1; - 1  
while (low <= high) { -  $O(\log n)$   
    mid = (low + high) / 2; - 1  
    if (arr[mid] == key) - 1  
    {  
        return mid; }  
    else if (arr[mid] < key) - 1  
    {  
        low = mid + 1; } - 1  
    else {  
        high = mid - 1; - 1  
    }  
}  
return - 1  
 $T(n) = O(\log n)$ 
```



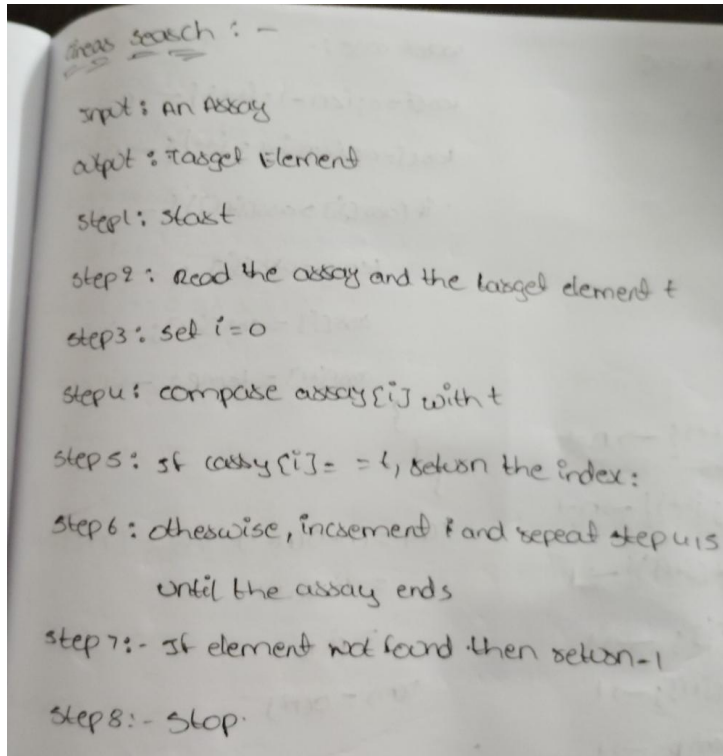
Worst case : Key is found at the last division / not found

```
low = 0; - 1
high = n-1; - 1
while (low <= high) {  $\rightarrow O(\log n)$ 
    mid = (low + high) / 2; - 1
    if (arr[mid] == key) - 1
    {
        return mid; }
    else if (arr[mid] < key) - 1
    {
        low = mid + 1; } - 1
    else {
        high = mid - 1; - 1
    }
}
return -1; - 1
T(n) =  $O(\log n)$ 
```



LINEAR SEARCH

ALGORITHM:



CODE:

```
#include <iostream>
using namespace std;

int main()
{
    int n, arr[100], key;

    cout << "Enter size of array: ";
    cin >> n;

    cout << "Enter array elements: ";
    for(int i = 0; i < n; i++)
        cin >> arr[i];

    cout << "Enter element to search: ";
    cin >> key;
```

```
for(int i = 0; i < n; i++)
{
    if(arr[i] == key)
    {
        cout << "Element found at position " << i + 1;
        return 0;
    }
}

cout << "Element not found";

return 0;
}
```

OUTPUT:

Output
Enter size of array: 5
Enter array elements: 34
67
23
65
1
Enter element to search: 24
Element not found

TIME COMPLEXITY:



Best case :

Linear search :-
Best case :-
element is found at the
first position
for (i=0; i<n; i++){
if (arr[i] == key){
return i;
}
T(n) = 1
T(n) = O(1)

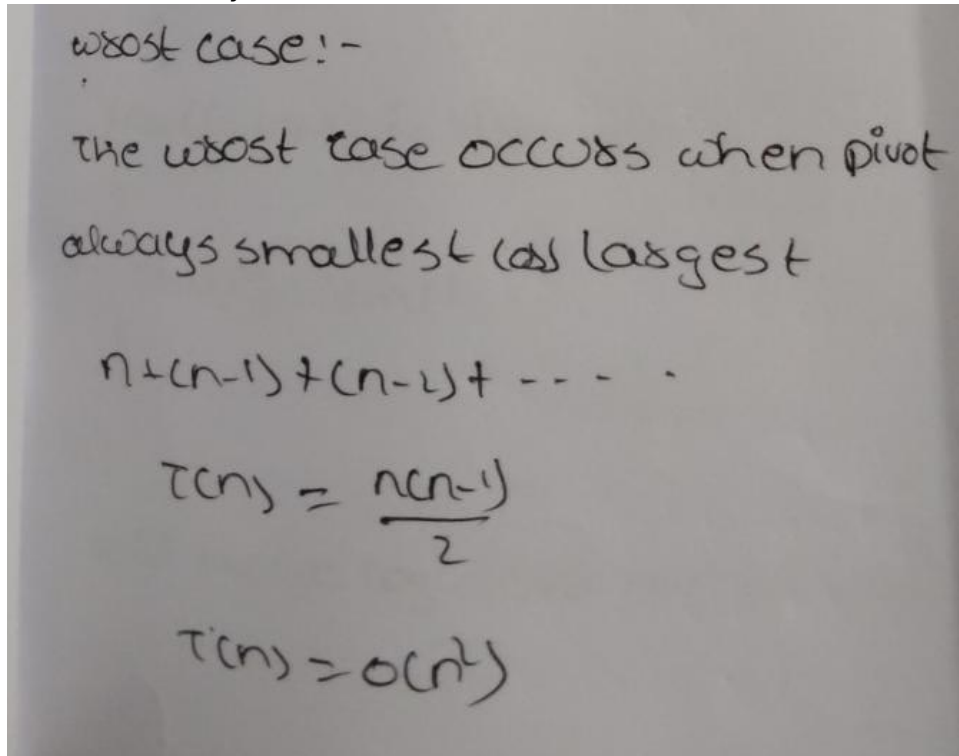


Average case:

Handwritten code for the average case of a linear search algorithm:

```
average case:-  
for(i=0; i<n; i++) — n  
{  
    if(arr[i] == key) — 1  
    {  
        return i; — 1  
    }  
}  
return -1; — 1  
  
Time = O(n)
```

worst case : key is not found at the last division



Conclusion:

From this implementation, I understood that the searching method depends on the type of data. Linear search is simple and checks elements one by one, while Binary search is faster because it repeatedly divides the sorted array into halves.